Matrix multiplication: Optimization Algorithms

Big Data (40386) - Solo assignment 2

Milosch Fürstenberg
https://github.com/miloschf/big-data-solo-assignment-2

Universidad de Las Palmas Gran Canaria (ULPGC)

Submission date: November 21, 2025

ULPGC Universidad de Las Palmas de Gran Canaria

Contents

List of Figures

List of abbrevations

ULPGC	Universidad de Las Palmas Gran Canaria
CPU	Central processing Unit
SIMD	Single Instruction, multiple data
ILP	Instruction level parallelism
CSR	Compressed sparse row
BCSR	Block compressed sparse row

ULPGC Universidad de Las Palmas de Gran Canaria

Abstract

Matrix multiplication is a core operation in scientific computing, data analytics, and machine learning, yet its classical $O(n^3)$ formula becomes prohibitively expensive for large matrices. This report investigates three established optimization techniques, Strassen's algorithm, loop unrolling, and cache tiling and evaluates their impact on computational performance and memory consumption in comparison to the classical approach. Additionally, the study analyzes how matrix sparsity influences runtime in both dense and sparse representations, including the Compressed Sparse Row (CSR) format.

Experimental results show that all tested optimizations substantially reduce runtime relative to classical multiplication, with cache tiling providing the most significant improvements across all matrix sizes. Since these optimizations can be combined, hybrid approaches represent a promising direction for achieving even higher performance. Memory measurements further indicate that all non-recursive algorithms, including the classical method, loop unrolling, and tiling, exhibit identical memory allocation behaviour, whereas Strassen's algorithm demonstrates dramatically higher memory usage, especially as matrix size increases, due to the growing number of intermediate matrices required at each recursion level.

The sparsity experiments reveal that sparsity only meaningfully accelerates classical matrix multiplication for small matrices; for larger matrices, the fixed cubic operation count dominates runtime. In contrast, CSR-based sparse multiplication exhibits a strong dependence on sparsity: runtimes decrease sharply as sparsity increases, whereas dense CSR matrices incur substantial overhead from indirect indexing and memory access patterns. These findings highlight the importance of selecting optimization strategies and storage formats based on matrix density and problem scale, and demonstrate that combining algorithmic and data-layout optimizations can yield substantial benefits for high-performance matrix computation.

1 Introduction

Matrix multiplication is one of the most fundamental operations in computational science, data analytics, and machine learning. It serves as a core component in a wide range of algorithms, from neural network training and dimensionality reduction to large-scale numerical simulations. The classical matrix multiplication algorithm, while conceptually simple, exhibits a cubic time complexity of $O(n^3)$, which rapidly becomes computationally expensive as matrix dimensions increase [1]. This computational bottleneck poses a significant challenge for Big Data applications and high-performance computing environments, where the efficient processing of large numerical datasets is essential.

To mitigate these performance constraints, several optimization strategies have been proposed and studied extensively in the literature. Algorithmic improvements such as Strassen's algorithm reduce the asymptotic complexity to approximately $O(n^{2.81})$ by decomposing the multiplication process into recursive subproblems [2]. In addition, hardware-oriented techniques such as loop unrolling and cache tiling improve instruction-level parallelism and memory locality, allowing modern CPUs to execute multiple operations per clock cycle efficiently [3]. These optimizations are especially relevant for matrix operations, which are often limited not by arithmetic throughput but by memory bandwidth and data reuse.

Another important dimension of optimization lies in the use of sparse matrices. In many real world applications, such as graph analytics, recommender systems, or finite element simulations, most matrix elements are zero. Exploiting this sparsity can dramatically reduce computational cost and memory footprint. Efficient sparse matrix multiplication (SpMM) algorithms rely on compact data structures such as Compressed Sparse Row (CSR) or Block Compressed Sparse Row (BCSR) formats and are optimized for emerging multicore and manycore platforms [4]. These optimizations emphasize minimizing data movement, improving cache utilization, and balancing computational workloads across cores.

This paper investigates and compares several optimized approaches to matrix multiplication, focusing on three algorithmic techniques Strassen's algorithm, loop unrolling, and cache tiling, as well as the implementation of sparse matrix multiplication. Through benchmarking and performance analysis, the study aims to evaluate the trade-offs between algorithmic efficiency, computational scalability, and memory behavior across different matrix sizes. By contrasting these optimized approaches with the classical algorithm, this research seeks to provide a comprehensive understanding of how optimization strategies influence performance in dense and sparse computational contexts within modern data-intensive applications.

2 Strassen's Algorithm

The Strassen algorithm, first introduced by Volker Strassen in 1969, represented a breakthrough in the study of fast matrix multiplication [2]. Unlike the classical algorithm, which performs $O(n^3)$ scalar multiplications, Strassen's method reduces the asymptotic complexity to approximately $O(n^{2.81})$ by exploiting a recursive divide-and-conquer strategy. This reduction was the first demonstration that matrix multiplication could be performed asymptotically faster than the cubic bound, and it laid the foundation for decades of research into fast linear algebra algorithms.

2.1 Algorithmic Principle

The key insight of Strassen's approach is that the product of two 2×2 block matrices can be computed using only seven multiplications instead of eight. This is achieved by introducing a sequence of intermediate matrices, M_1 through M_7, which are linear combinations of submatrices of A and B. These intermediate results are then recombined through addition and subtraction to form the resulting matrix C. By recursively applying this decomposition to sub-blocks, the total number of arithmetic operations grows at a rate proportional to $n^{\log_2 7}$ rather than n^3. Although the algorithm introduces additional additions and subtractions, these are asymptotically less costly than the reduction in multiplications.

2.2 Computational Trade-offs

While Strassen's algorithm reduces theoretical complexity, its practical efficiency depends on multiple implementation factors. For small matrix sizes, the constant overhead from recursive calls and extra additions can outweigh the benefits of fewer multiplications [5]. Moreover, the algorithm is less numerically stable than the classical approach due to the increased number of arithmetic operations and potential rounding errors in floating-point computation [6]. These stability issues make Strassen's method less suitable for high-precision scientific applications without further numerical safeguards.

Despite these challenges, Strassen's algorithm remains a cornerstone in the field of computational linear algebra. It has inspired numerous subsequent developments, such as the Coppersmith Winograd algorithm and recent practical adaptations for parallel and cache-optimized architectures [7], [8]. In modern high-performance computing contexts, hybrid implementations often switch between the classical and Strassen's algorithm depending on matrix size and hardware characteristics to balance accuracy and performance [9].

3 Loop Unrolling

Loop unrolling is a classical compiler and algorithmic optimization technique that aims to reduce the overhead of loop control and increase instruction-level parallelism (ILP) during program execution [10]. In the context of matrix multiplication, where the same arithmetic operation pattern is executed repeatedly over large data arrays, loop unrolling can yield substantial performance improvements by minimizing branching operations and exposing additional opportunities for parallel execution within the processor pipeline.

3.1 Concept and Mechanism

In its basic form, loop unrolling replaces a loop body with multiple copies of itself, thereby decreasing the total number of loop iterations and the associated conditional and increment instructions. For example, instead of performing one multiplication and addition per iteration, the loop body can be replicated to handle multiple elements per iteration. This transformation increases the number of independent arithmetic operations that can be executed concurrently, allowing modern superscalar processors to better utilize their multiple functional units [3]. In addition, it reduces instruction-fetch and branch-prediction overhead, both of which can be limiting factors in performance-critical numerical computations.

3.2 Impact on Matrix Multiplication

For dense matrix multiplication, loop unrolling improves performance primarily by enhancing data reuse within the CPU cache and reducing the number of control instructions per arithmetic operation. When applied to the innermost multiplication loops, unrolling allows compilers to schedule instructions more efficiently, improving pipeline utilization and register allocation [11]. On architectures supporting SIMD or vectorized operations, unrolled loops can also facilitate automatic vectorization, enabling multiple elements of a row or column to be processed in parallel [3], [12]. However, excessive unrolling may lead to code bloat and register pressure, which can degrade performance if the working set exceeds cache capacity or register availability.

3.3 Implementation Considerations

The effectiveness of loop unrolling depends on the target architecture, compiler optimization capabilities and problem size. In high-performance computing environments, manual or compiler-assisted unrolling is often combined with additional optimizations such as tiling and blocking to exploit memory hierarchy characteristics [13]. For matrix multiplication in particular, empirical studies have shown that moderate unrolling factors (typically between 2× and 8×) provide the best trade-off between code

compactness and throughput [4]. When used together with cache tiling and instruction prefetching, loop unrolling contributes significantly to reducing memory stalls and improving arithmetic intensity.

3.4 Relevance in Optimization Frameworks

Loop unrolling represents an important bridge between compiler-level optimization and algorithmic design. While it does not alter the theoretical complexity of matrix multiplication, it can achieve tangible runtime improvements by aligning computation with modern CPU microarchitectures. In this study, loop unrolling serves as a representative of low-level instruction-level optimization techniques, complementing higher-level algorithmic strategies such as Strassen's recursive multiplication and cache-aware tiling that will be discussed in the next section.

4 Tiling / Blocking Optimization

Cache tiling, also known as blocking, is a memory optimization technique designed to improve data locality and cache reuse in computationally intensive operations such as matrix multiplication [3]. The method divides large matrices into smaller submatrices or tiles, which fit into the processor's cache hierarchy, allowing repeated access to cached data and reducing costly main-memory transfers. Because modern CPUs exhibit a significant latency gap between cache and main memory, cache-efficient algorithms such as tiling play a critical role in achieving near-peak performance for dense linear algebra computations.

4.1 Concept and Algorithmic Basis

The fundamental idea of cache tiling is to partition the iteration space of nested loops into smaller blocks so that a portion of the data being processed can reside entirely in the faster levels of the cache. For a classical three-nested-loop matrix multiplication, this involves splitting each dimension (i, j, k) into tile sizes (T_i, T_j, T_k), resulting in block-wise operations on submatrices. Each block is multiplied and accumulated before moving to the next, ensuring temporal and spatial data locality [11]. The tile size is typically chosen empirically or through hardware-aware tuning so that the working set fits within the L1 or L2 cache.

4.2 Performance Characteristics

Tiling substantially increases the arithmetic intensity of matrix multiplication, which is the ratio of computations to memory accesses, by ensuring that elements of A and B loaded into cache are reused multiple times before eviction. This results in a lower memory-bandwidth requirement and improved pipeline utilization. Empirical studies show that cache-blocked algorithms can achieve performance

close to the theoretical peak of the underlying hardware [14], [15]. Moreover, tiling enhances the effectiveness of other optimizations, such as loop unrolling or vectorization, by structuring memory accesses into contiguous patterns that align with cache lines and SIMD registers.

4.3 Implementation and Practical Considerations

The optimal tile size depends on hardware parameters including cache capacity, associativity, and memory-bus bandwidth. Automatic tuning frameworks such as ATLAS and OpenBLAS determine these parameters empirically to select cache-optimal block sizes at runtime [15]. More recent research on communication-avoiding algorithms demonstrates that cache tiling can be extended hierarchically across multiple cache levels and even distributed memory systems [7]. In multicore environments, tiles can be allocated to threads independently, allowing parallel processing with minimal data contention when combined with NUMA-aware scheduling.

4.4 Relevance for Matrix Multiplication Optimization

In the context of this study, tiling represents a key optimization bridging algorithmic design and architectural efficiency. While it does not reduce the asymptotic complexity of matrix multiplication, it maximizes effective memory throughput and minimizes latency penalties, both of which are dominant in modern computing architectures. By combining tiling with complementary optimizations such as loop unrolling and Strassen's recursive decomposition, it is possible to achieve substantial performance improvements for large matrix operations on contemporary multicore processors.

5 Matrix Sparsity and its Influence on Multiplication Performance

Matrix sparsity refers to the proportion of zero-valued elements within a matrix. While many real-world datasets, such as graphs, recommendation systems, or finite-element simulations, exhibit sparsity levels exceeding 90%, the computational implications of sparsity in matrix multiplication are non-trivial. From a purely theoretical perspective, sparsity should not influence runtime in a naïve dense matrix multiplication algorithm, since the algorithmic complexity remains $O(n^3)$ regardless of how many entries are zero. Each element is multiplied and accumulated independently of its value, and multiplication by zero is still counted as a constant-time arithmetic operation.

However, in practice, sparsity can significantly affect performance due to microarchitectural characteristics of modern CPUs. Multiplying by zero often triggers fast-path optimizations in hardware, consuming fewer CPU cycles and sometimes avoiding unnecessary register operations or partial pipeline stages [16]. Furthermore, sparse data tends to exhibit irregular memory-access patterns, which can either hinder or accelerate computation depending on how the underlying matrix is stored.

5.1 Sparse Storage Formats

To exploit sparsity more explicitly, numerous specialized storage formats have been developed. The most widely used is the *Compressed Sparse Row* (CSR) format, which stores only the non-zero values of a matrix alongside their column indices and row pointers. CSR drastically reduces the memory footprint of sparse matrices and improves cache locality by enabling sequential access to non-zero elements [17]. This reduction in memory traffic is a dominant factor in performance, since sparse matrix-vector and matrix-matrix operations are typically memory-bound rather than compute-bound [4].

Sparse formats such as CSR, CSC (Compressed Sparse Column), COO (Coordinate Format), and block-structured variants (e.g., BCSR) allow multiplication algorithms to skip over implicit zeros entirely. As a result, the number of arithmetic operations decreases from $O(n^3)$ to $O(\text{nnz} \cdot n)$ or even lower depending on sparsity structure. For large, highly sparse matrices, this leads to dramatic performance improvements and can shift performance bottlenecks from floating-point units to memory bandwidth and pointer-chasing overhead.

5.2 Practical Implications for Matrix Multiplication

Despite these benefits, sparse matrix multiplication presents unique challenges. Unlike dense multiplication, sparse operations often suffer from low spatial locality due to indirect indexing, making prefetching and cache optimization more difficult [18]. Performance therefore depends heavily on matrix structure, sparsity distribution, and whether non-zero entries form patterns that can be exploited by block-based or tile-based sparse formats.

Summarized, although sparsity does not theoretically alter the computational complexity of dense matrix multiplication, practical considerations, including hardware-level zero-multiplication optimizations, reduced memory movement via sparse storage schemes, and the ability to skip computations, can yield substantial performance differences. Thus, sparsity plays an important role in modern high-performance linear algebra, particularly when combined with formats such as CSR that are designed to minimize overhead and leverage architectural strengths.

6 Benchmarking

6.1 Benchmarking Device

All benchmarking experiments were conducted on a single mobile computing platform to ensure consistency and reproducibility of performance measurements. The system used for all implementations and benchmarks was an HP Envy x360 15-fh0xxx 2-in-1 laptop equipped with an **AMD Ryzen 7 7730U processor featuring 8 physical cores and 16 logical threads, operating at a base fre-**

quency of 2.0 GHz. The processor includes integrated Radeon Graphics and is built upon the Zen 3 architecture, optimized for energy-efficient multithreaded workloads.

The device was configured with **16 GB DDR4-3200 MHz of dual-channel system memory** and a 512 GB NVMe SSD for storage. The operating system environment was **Windows 11 Home (64-bit)**, running under UEFI BIOS version F.10 (dated July 23, 2024). All experiments were executed under identical software and power conditions, with background processes minimized to reduce system noise and ensure stable performance readings. Thermal and power management settings were fixed to the balanced power profile provided by the manufacturer to prevent dynamic frequency scaling from affecting measurements.

All matrix multiplication benchmarks, both classical and optimized variants, were executed on this configuration to maintain methodological consistency across all algorithmic comparisons.

Table 1: Hardware and System Specifications of Benchmarking Device

Component	**Specification**
System Model	HP Envy x360 15-fh0xxx (2-in-1)
Processor	AMD Ryzen 7 7730U (8 Cores / 16 Threads, 2.0 GHz)
Graphics	Integrated Radeon Graphics
Memory	16 GB DDR4-3200 (Dual Channel)
Storage	512 GB NVMe SSD
BIOS Version	Insyde F.10 (23.07.2024)
Operating System	Windows 11 Home 64-bit (Build 26100.1)
Architecture	x64-based System (UEFI Boot Mode)
Thermal Mode	Balanced Power Profile

6.2 Benchmark Implementation

The benchmarking methodology was designed to ensure reliable, reproducible, and architecture-aware performance measurements for all matrix multiplication algorithms evaluated in this study. Several measurement techniques were employed depending on the type of algorithm and metric, following best practices in empirical performance analysis.

Runtime Measurements Using JMH

For all dense matrix multiplication algorithms: the classical approach, Strassen's algorithm, loop unrolling, and cache tiling, the runtime measurements were performed using the Java Microbenchmark Harness (JMH). JMH is specifically designed for microarchitectural benchmarking on the JVM and provides statistically robust measurements by handling warm-up phases, JVM optimizations, dead-code elimination, and measurement noise.

The JMH configuration used in this work employed:

- AverageTime mode to measure mean execution time per benchmark invocation,

- Millisecond precision as output time unit,
- two warm-up iterations of one second each to allow the JVM to reach steady state,
- five measurement iterations of one second each,
- and a single fork to avoid excessive JVM startup overhead.

Each benchmark trial generated two randomly initialized matrices of size

$$n \in \{128, 256, 512, 1024, 4096\},$$

which were then multiplied using the four algorithmic variants under evaluation. The dataset initialization and parameter management were encapsulated in a dedicated JMH `@State` object. This setup ensured fair and isolated benchmarking conditions across all algorithmic configurations. [19]

Memory Measurements via Byte Allocation

To assess memory consumption during dense matrix multiplication, the benchmarking framework relied on byte allocation tracking. This method records the number of allocated bytes per benchmark invocation, providing an accurate approximation of heap activity for each algorithm.

Since Strassen's algorithm and tiling techniques allocate additional temporary structures during recursive subdivision and block operations, byte-allocation metrics offer valuable insights into each algorithm's memory behavior. The measurement captures JVM-level heap traffic and therefore reflects the true memory overhead of the Java-based implementations.

Sparse Matrix Runtime Measurements

Sparse matrix multiplication was benchmarked separately using:

```
System.nanoTime()
```

which provides nanosecond-resolution timestamps suitable for short, computation-dense operations typical in sparse matrix workloads.

Each sparse multiplication operation executed a single multiplication of a randomly generated sparse matrix under different sparsity levels, ensuring realistic performance comparisons between dense and sparse approaches. The `nanoTime()` measurements were repeated multiple times and averaged to mitigate jitter caused by OS scheduling and memory-access irregularities. [20]

6.3 Algorithmic optimization Results

The runtime results presented in Figures 1 and 2 clearly illustrate the substantial performance gap between the classical matrix multiplication algorithm and its optimized counterparts. For the largest tested matrix size of 4096×4096, the classical implementation is approximately 30 times slower than even the slowest optimized variant. This drastic difference is expected, as the naïve algorithm performs a strict $O(n^3)$ number of arithmetic operations without leveraging any hardware-level or cache-aware optimizations [14].

Among the optimized techniques, cache tiling achieves the best performance across all matrix sizes. By restructuring computations into cache-friendly blocks, tiling significantly increases data locality and reduces memory bandwidth pressure, enabling much higher effective throughput. Following tiling, Strassen's algorithm demonstrates the next-best performance, benefiting from its reduced asymptotic complexity of $O(n^{2.81})$ compared to the cubic baseline. Loop unrolling performs consistently but lags behind the other two optimizations, as its primary improvements stem from instruction-level parallelism rather than algorithmic or cache-level enhancements.

It is important to note that these optimizations are not mutually exclusive. Hybrid approaches, such as combining Strassen's recursion with cache tiling or unrolled micro-kernels, have been shown to yield even greater performance gains on modern architectures by simultaneously improving arithmetic efficiency and memory locality [7]. Thus, while each individual optimization provides substantial speedups over the classical approach, a combined strategy could potentially outperform all standalone implementations observed in this study.

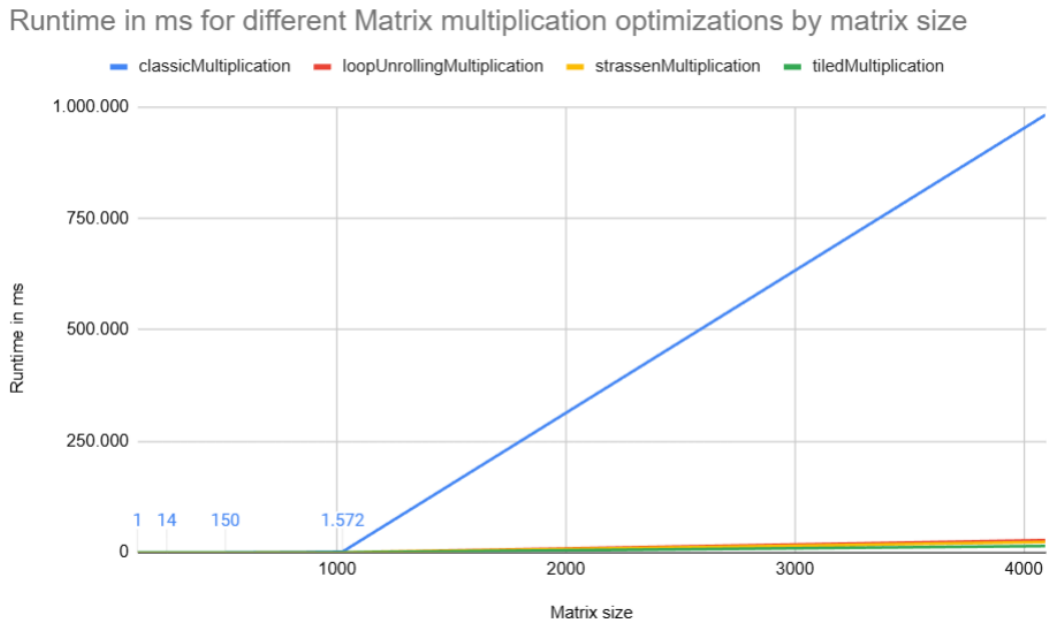

Figure 1: Algorithmic runtimes with the classic matrix multiplication as reference

The memory allocation results shown in Figure 3 illustrate a clear difference between the classical,

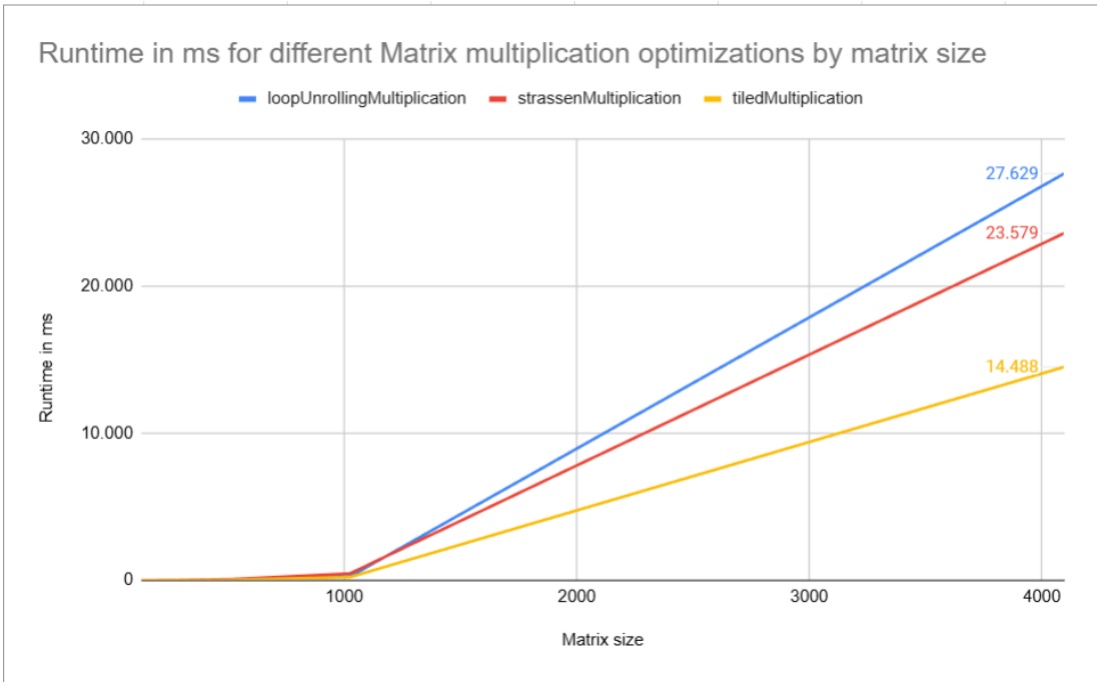

Figure 2: Algorithmic runtimes without the classic matrix multiplication

loop-unrolled, and tiled matrix multiplication algorithms compared to Strassen's algorithm. For all three non-recursive approaches, the allocated bytes remain nearly identical across matrix sizes. This is expected: all of these algorithms operate directly on the input matrices and produce the output matrix without allocating large auxiliary structures. Their memory usage is therefore dominated by the storage requirement for the three $n \times n$ dense matrices, resulting in the well-known $O(n^2)$ space complexity.

Strassen's algorithm, however, exhibits significantly higher memory allocation that increases rapidly with matrix size. This behaviour stems from Strassen's recursive decomposition strategy. At each recursion level, the algorithm computes seven submatrix products and several additional intermediate matrices produced by additions and subtractions of sub-blocks. These temporary matrices are required to store intermediate results before the final recursive combination can be performed [6]. Because Strassen reduces the number of multiplications at the expense of more additions, the total amount of auxiliary data grows across recursion levels.

Mathematically, fast matrix multiplication algorithms such as Strassen's are known to require extra workspace proportional to the sum of temporary subproblem sizes [21]. As the recursion proceeds, each level allocates multiple $n/2 \times n/2$ intermediate matrices, resulting in a cumulative memory footprint substantially larger than that of classical multiplication. This explains the steep upward trend in allocated bytes visible in the measurements and highlights an intrinsic trade-off of Strassen's algorithm: reduced arithmetic cost but noticeably increased memory consumption.

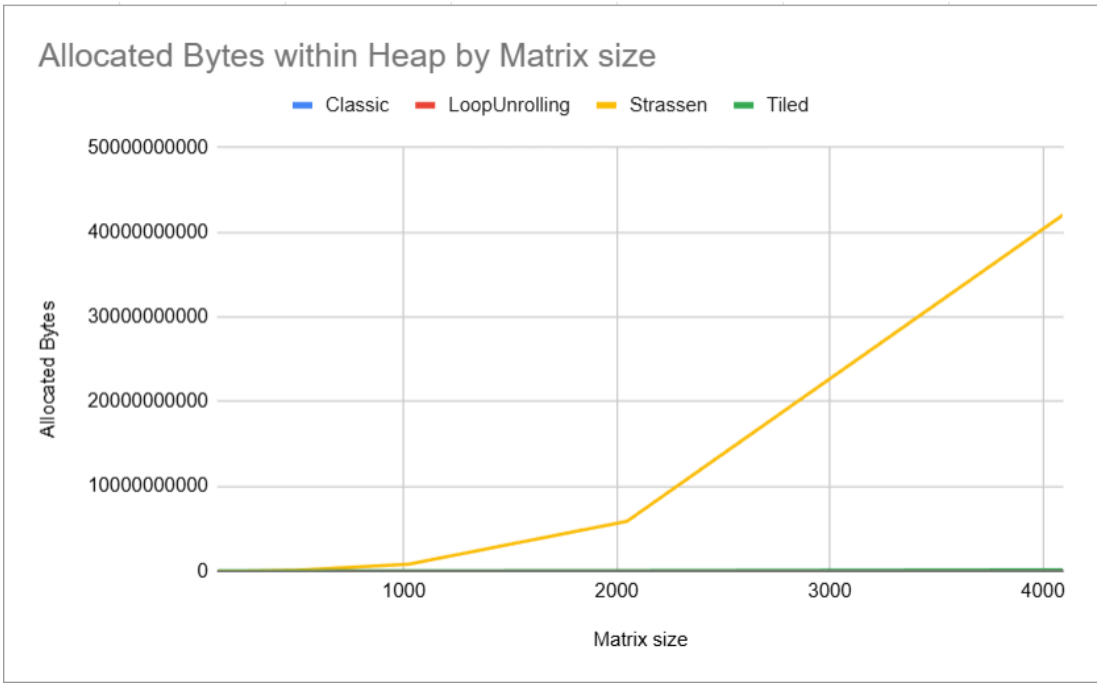

Figure 3: Byte Allocation across different algorithmic optimizations

6.4 Sparsity runtime results

The sparsity experiments shown in Figures 4–7 reveal a distinct trend: for smaller matrices, increasing sparsity leads to noticeably reduced runtime in the classical matrix multiplication algorithm, whereas for larger matrices the effect becomes progressively marginal. For example, at a matrix size of 256×256, the runtime decreases from 11 ms at a sparsity of 0.25 to roughly 2–3 ms for sparsity levels of 0.9 and above. In contrast, for larger matrices such as 4096×4096, the runtime remains nearly constant across sparsity levels, fluctuating only slightly around 25–26 s.

This behaviour can be explained by the operational characteristics of the classical $O(n^3)$ dense matrix multiplication algorithm. Although sparsity changes the distribution of non-zero values, the algorithm still performs the same number of arithmetic operations regardless of how many entries are zero. Each loop iteration executes a multiplication and an addition, even when multiplying by zero. Thus, for large matrices the computational workload is dominated by the fixed cubic number of iterations, and sparsity has little impact on total runtime [16].

For smaller matrices, however, several microarchitectural effects amplify the influence of sparsity. Modern processors often implement fast-path optimizations for multiplication by zero, allowing the floating-point unit to skip or simplify parts of the execution pipeline, which reduces the effective instruction latency [22]. Furthermore, smaller matrices reside more comfortably within cache hierarchies. At high sparsity levels, a larger proportion of the data accessed during computation may be zeros, which leads to more predictable memory access patterns and fewer cache stalls, resulting in shorter runtimes.

As matrix sizes grow, the algorithm becomes increasingly bound by memory traffic and the

sheer number of loop iterations rather than by individual arithmetic optimizations or cache effects. This is consistent with established findings that dense matrix multiplication becomes predominantly compute-bound at large scales, with runtime dominated by the total number of operations rather than data values [23]. Consequently, the runtime curves converge for higher matrix sizes, and sparsity ceases to have a meaningful impact on performance.

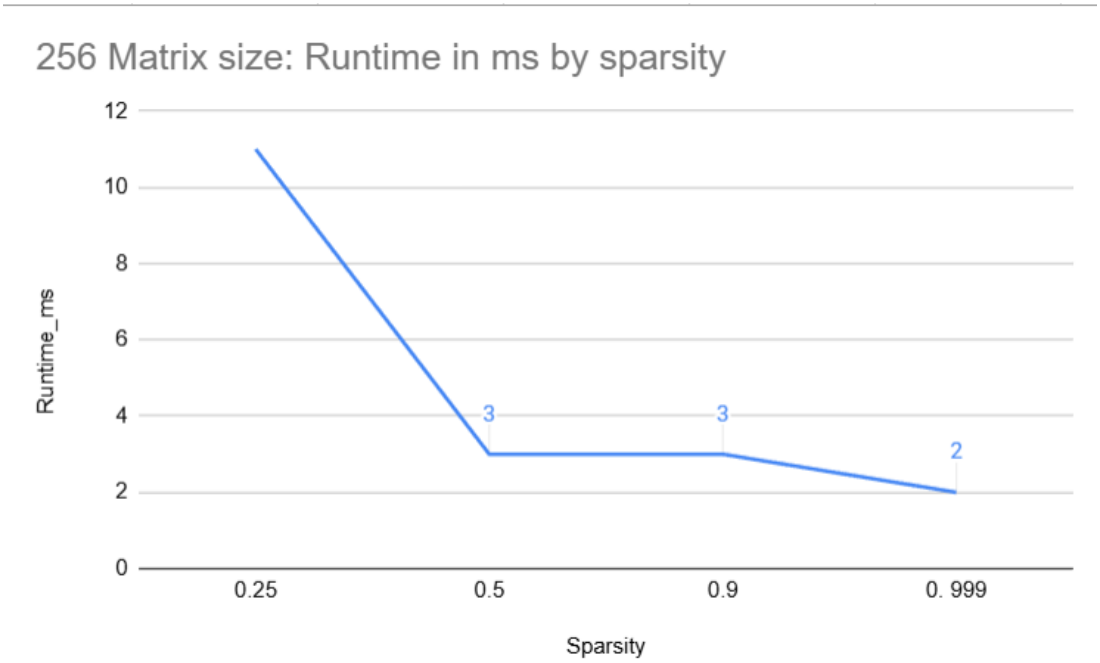

Figure 4: 256x256 Matrix multiplication runtime by sparsity

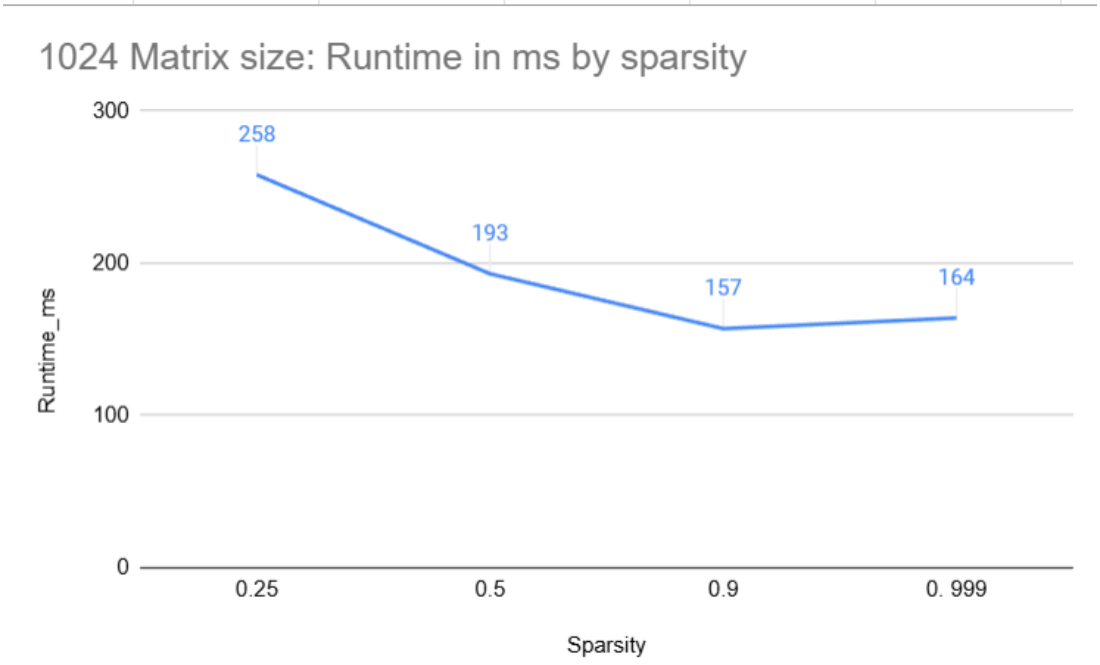

Figure 5: 1024x1024 Matrix multiplication runtime by sparsity

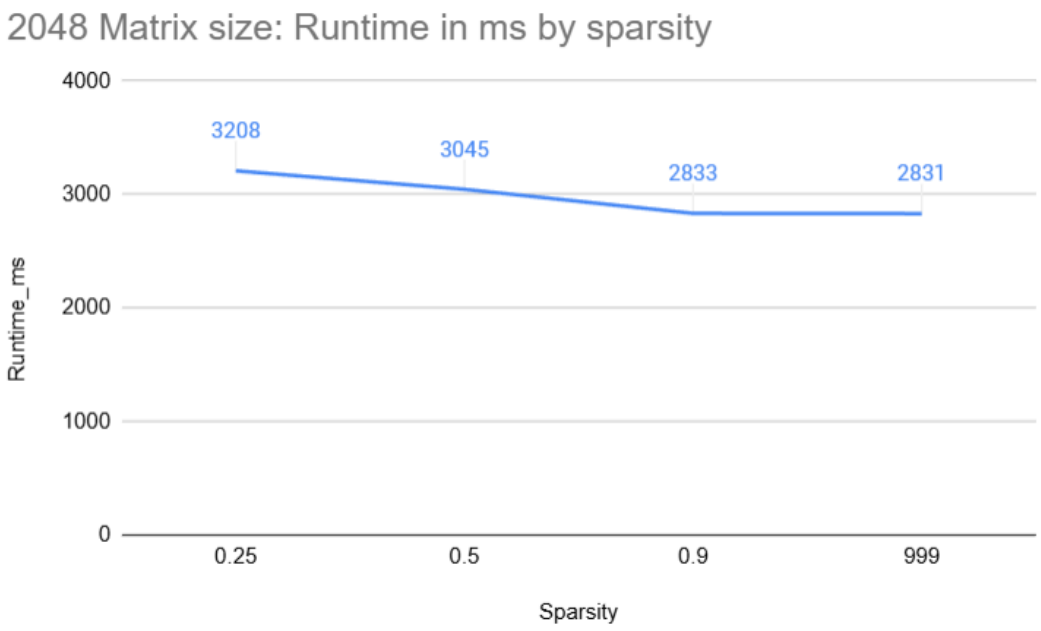

Figure 6: 2048x2048 Matrix multiplication runtime by sparsity

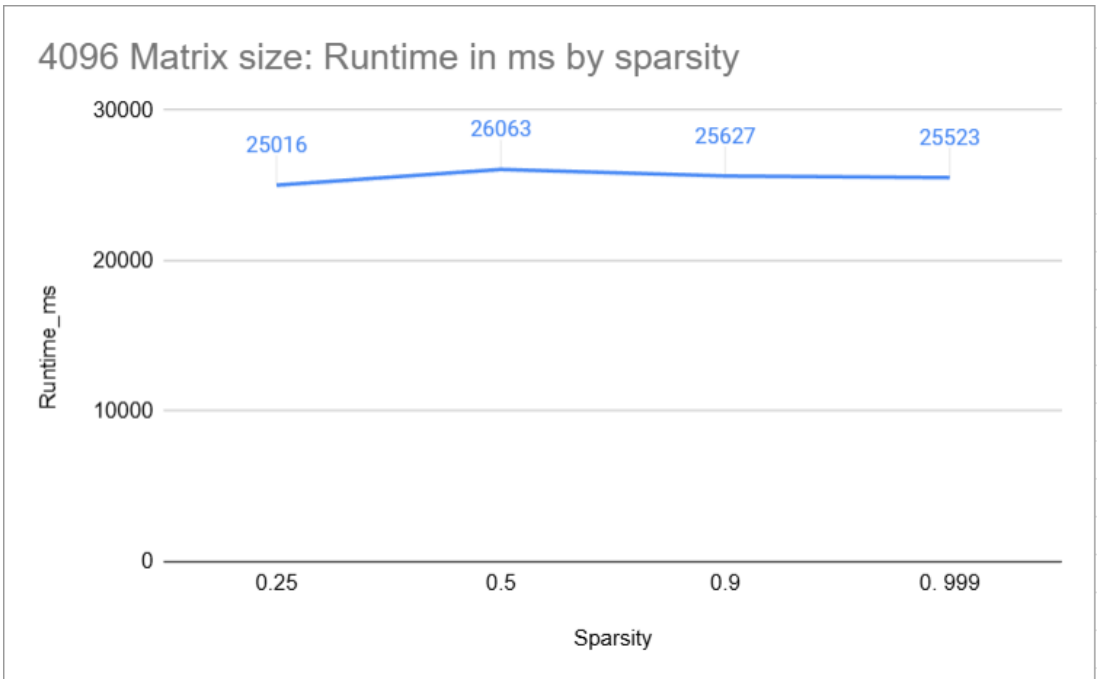

Figure 7: 4096x4096 Matrix multiplication runtime by sparsity

The results in Figure 8 demonstrate how dramatically the performance of sparse matrix multiplication improves when using the Compressed Sparse Row (CSR) format. For a 2048×2048 matrix, the runtime decreases steeply as sparsity increases: from 4801 ms at a sparsity of 0.25 down to only 43 ms at a sparsity of 0.999. This reduction is far more pronounced than in the dense classical matrix multiplication results for the same matrix size, where sparsity produced only marginal improvements. The reason is that, unlike the classical $O(n^3)$ approach, CSR-based multiplication performs opera-

tions *only* on the explicitly stored non-zero entries. As sparsity grows, the number of stored values (nnz) shrinks accordingly, and the computational cost approaches $O(\text{nnz})$ rather than $O(n^3)$ [17]. Because memory traffic is the dominant performance factor in sparse algebra, eliminating zero-valued elements leads directly to reduced memory accesses and therefore lower runtime [4].

These results indicate that CSR multiplication becomes increasingly advantageous for large matrices as sparsity increases. When most entries are zero, CSR avoids unnecessary multiplications and memory loads, yielding orders-of-magnitude speedups compared to dense multiplication. However, the trend reverses for dense or near-dense matrices where CSR multilpication runs longer than the classical 2048x2048 matrix multiplication. In such cases, CSR introduces substantial overhead: indirect indexing, pointer traversal, and scattered memory accesses, all of which lead to reduced cache efficiency and slower runtime compared to dense matrix multiplication [18]. CSR is therefore most effective when sparsity is very high and becomes suboptimal as matrices approach dense structure.

Overall, these findings confirm that sparse matrix formats such as CSR should be preferred for large-scale matrix operations when sparsity is substantial, whereas dense multiplication remains more efficient for matrices with high density.

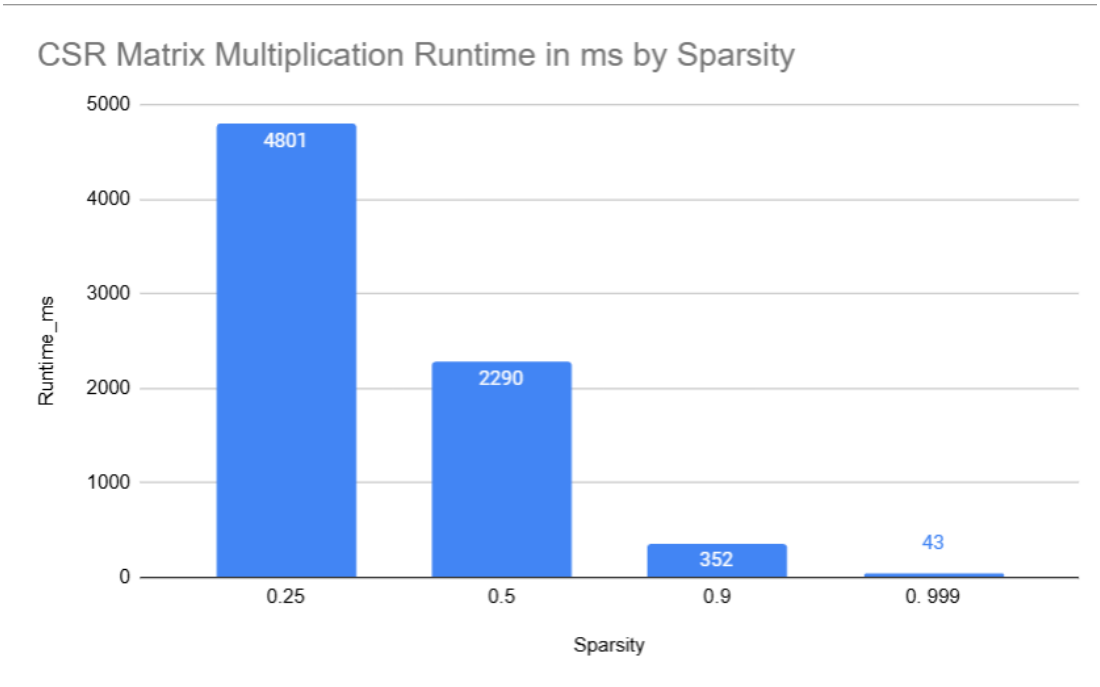

Figure 8: CSR 2048x2048 Matrix multiplication runtime across different sparsities

7 Conclusion

The results of this study demonstrate that algorithmic and architectural optimizations can substantially improve the performance of matrix multiplication. Among the evaluated methods, tiling achieved the most significant reduction in runtime across all matrix sizes, primarily due to its ability to enhance

cache locality and reduce memory bandwidth pressure. Loop unrolling and Strassen's algorithm also provided considerable speedups over the classical implementation, although to a lesser degree. Importantly, these optimizations are not mutually exclusive; hybrid strategies combining tiling, loop unrolling, and Strassen's recursion may yield even greater performance gains and therefore represent a promising direction for further investigation.

In terms of memory consumption, all non-recursive algorithms, including the classical method, loop unrolling and tiling exhibited identical memory allocation patterns. This behaviour is expected given that all three approaches operate directly on the input matrices without introducing large auxiliary data structures. In contrast, Strassen's algorithm requires substantially more memory, and this overhead grows rapidly with increasing matrix size. The recursive decomposition of the algorithm introduces numerous temporary matrices at each recursion level, leading to a dramatically higher memory footprint compared to the other methods.

The sparsity experiments revealed that sparsity only meaningfully impacts runtime in the classical algorithm for smaller matrices. As matrix dimensions grow, the fixed $O(n^3)$ operation count dominates, rendering sparsity increasingly irrelevant. However, when matrices are stored in the Compressed Sparse Row (CSR) format, the performance characteristics change fundamentally: runtimes become significantly shorter for highly sparse matrices and substantially longer for dense matrices. CSR multiplication therefore offers substantial benefits for large, sparsely populated matrices but is inefficient for dense ones due to indexing overhead and reduced cache locality.

Overall, the findings highlight the importance of selecting appropriate optimization strategies based on matrix size, density, and target architecture. While tiling provides the most robust improvement for dense matrices, sparse data structures such as CSR are essential for efficiently handling large and sparse workloads. Future work may consider combining multiple optimizations and extending the evaluation to parallel and distributed environments to further enhance scalability and performance.

List of sources

[1] Thomas H. Cormen et al. *Introduction to Algorithms*. 3rd. MIT Press, 2009. ISBN: 9780262033848.

[2] Volker Strassen. "Gaussian Elimination is not Optimal". In: *Numerische Mathematik* 13.4 (1969), pp. 354–356. DOI: 10.1007/BF02165411.

[3] Monica S. Lam, Edward E. Rothberg, and Michael E. Wolf. "The Cache Performance and Optimizations of Blocked Algorithms". In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*. ACM, 1991, pp. 63–74. DOI: 10.1145/106972.106981.

[4] Samuel Williams et al. "Optimization of Sparse Matrix-Vector Multiplication on Emerging Multicore Platforms". In: *Proceedings of the 2007 ACM/IEEE Conference on Supercomputing (SC '07)*. IEEE/ACM, 2007, pp. 1–12. DOI: 10.1145/1362622.1362674.

[5] Steven Huss-Lederman et al. "Parallel Implementation of Strassen's Matrix Multiplication Algorithm for Shared-Memory Multiprocessors". In: *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing (SC '91)*. IEEE, 1991, pp. 1–10. DOI: 10.1145/125826.125932.

[6] Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. 2nd. SIAM, 2002. DOI: 10.1137/1.9780898718027.

[7] Grey Ballard et al. "Communication-Optimal Parallel and Sequential Matrix Multiplication". In: *SIAM Journal on Scientific Computing* 34.2 (2012), pp. C110–C141. DOI: 10.1137/110848244.

[8] Juha Kärppä and François Le Gall. "Fast Matrix Multiplication: Limitations and Practical Improvements". In: *Journal of the ACM* 68.3 (2021), pp. 1–29. DOI: 10.1145/3447572.

[9] Srinivas Aluru. *Handbook of Parallel Computing: Models, Algorithms, and Applications*. Chapman and Hall/CRC, 2020. DOI: 10.1201/9780429266281.

[10] Randy Allen and Ken Kennedy. *Optimizing Compilers for Modern Architectures: A Dependence-based Approach*. Morgan Kaufmann, 2002. ISBN: 9781558602860.

[11] Michael E. Wolf and Monica S. Lam. "Data Locality and Optimization of Loops". In: *ACM Transactions on Programming Languages and Systems* 16.4 (1994), pp. 924–973. DOI: 10.1145/183432.183433.

[12] Krste Asanovic et al. *The Landscape of Parallel Computing Research: A View from Berkeley*. Tech. rep. UCB/EECS-2006-183. University of California, Berkeley, 2006.

[13] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 5th. Morgan Kaufmann, 2012. ISBN: 9780123838728.

[14] Kazushige Goto and Robert A. van de Geijn. “Anatomy of High-Performance Matrix Multiplication”. In: *ACM Transactions on Mathematical Software* 34.3 (2008), pp. 1–25. DOI: 10.1145/1356052.1356053.

[15] R. Clinton Whaley and Jack J. Dongarra. “Automatically Tuned Linear Algebra Software”. In: *Proceedings of the 1998 ACM/IEEE Conference on Supercomputing (SC ’98)* (1998), pp. 1–27. DOI: 10.1145/509058.509096.

[16] Jack Dongarra. “Performance of Various Computers Using Dense Linear Algebra”. In: *Concurrency and Computation: Practice and Experience* 12.9 (2000), pp. 771–792. DOI: 10.1002/cpe.567.

[17] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2003. DOI: 10.1137/1.9780898718003.

[18] Richard Vuduc, James Demmel, and Katherine Yelick. “OSKI: A Library of Automatically Tuned Sparse Matrix Kernels”. In: *Scientific Discovery through Advanced Computing (SciDAC) Conference*. IOP, 2005. DOI: 10.1088/1742-6596/16/1/062.

[19] OpenJDK. *Java Microbenchmark Harness (JMH) Official Documentation*. https://openjdk.org/projects/code-tools/jmh/. Accessed: 2025-02-01. 2024.

[20] Oracle Corporation. *Java SE Documentation: System.nanoTime()*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/System.html#nanoTime(). Accessed: 2025-02-01. 2024.

[21] Dario Bini and Gianfranco Lotti. “On the Complexity of Fast Matrix Multiplication”. In: *Advances in Computers*. Vol. 20. Academic Press, 1980, pp. 75–112. DOI: 10.1016/S0065-2458(08)60194-1.

[22] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. Tech. rep. CRC Press, 2010.

[23] Samuel Williams, Andrew Waterman, and David Patterson. “Roofline: An Insightful Visual Performance Model for Multicore Architectures”. In: *Proceedings of the 2009 International Conference on High Performance Computing, Networking, Storage and Analysis (SC09)*. IEEE, 2009, pp. 1–11. DOI: 10.1145/1654059.1654074.

Appendix

AI protocol

- Chatgpt 5.1 was used to research sources and rewrite text or texturize bullet points
- Writeful AI was used to check grammar mistakes and rewrite text to sound more scientific/ professional
- Claude AI was used to debug code and explain code
- Github Coplilot was used to debug code